

DOKUMENTASI BE TAHAP 3

Proyek: Lost & Found Service – Dragon Waterpark

Nama: Rafif Danu Saputra

Npm : 23312079

1. Menyelesaikan Alur Claim Barang

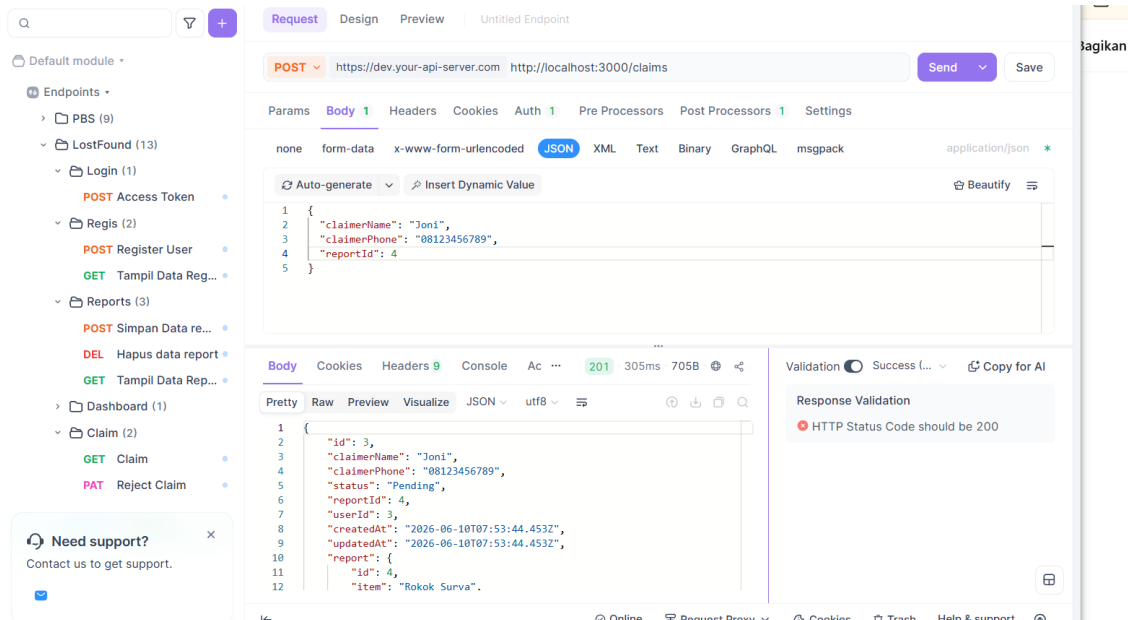
Yang saya kerjakan :

- Menambahkan hubungan antara data claim dan akun user.
- Menyimpan userId secara otomatis dari JWT saat membuat claim.
- Memastikan admin dapat melakukan approve dan reject claim.
- Memastikan status report berubah menjadi Claimed ketika claim disetujui.

Hasil:

- Create Claim berhasil.
- Approve Claim berhasil.
- Reject Claim berhasil.
- Sinkronisasi status report berjalan dengan baik.

Create Claim Berhasil :



Approve Claim

The screenshot shows the REST Client interface with a sidebar on the left containing a list of APIs. The main panel displays a PATCH request to `http://localhost:3000/claims/3/status`. The request body is a JSON object with `"status": "Approved"`. The response is a JSON object with details about the claim, including `"id": 3`, `"claimName": "Jon1"`, `"claimPhone": "08123456789"`, `"status": "Approved"`, `"reportId": 4`, `"userId": 3`, `"createdAt": "2026-06-10T07:53:44.453Z"`, `"updatedAt": "2026-06-10T07:56:071Z"`, and a `"report"` object with `"id": 4` and `"item": "Rokok Surva"`. The status bar indicates a successful response (200) and the request passed validation.

APIs: main

Default module

Endpoints

- PBS (9)
- LostFound (13)
- Login (1)
- POST Access Token
- Regs (2)
- POST Register User
- GET Tampil Data Reg...
- Reports (3)
- POST Simpan Data re...
- DEL Hapus data report
- GET Tampil Data Rep...
- Dashboard (1)
- Claim (2)
- GET Claim
- PAT Reject Claim

Need support? Contact us to get support.

Request: PATCH `http://localhost:3000/claims/3/status`

Params: Body 1 Headers Cookies Auth 1 Pre Processors Post Processors 1 Settings

none form-data x-www-form-urlencoded JSON XML Text Binary GraphQL msgpack application/json

Auto-generate Insert Dynamic Value Beautify

```
1 {
2   "status": "Approved"
3 }
```

Body Cookies Headers 9 Console Actual ... 200 329ms 701B Validation Success (200) Passed

Pretty Raw Preview Visualize JSON utf8

```
1 {
2   "id": 3,
3   "claimName": "Jon1",
4   "claimPhone": "08123456789",
5   "status": "Approved",
6   "reportId": 4,
7   "userId": 3,
8   "createdAt": "2026-06-10T07:53:44.453Z",
9   "updatedAt": "2026-06-10T07:56:071Z",
10  "report": {
11    "id": 4,
12    "item": "Rokok Surva".
13  }
14 }
```

Design-first Mode Request-first Mode Online Request Proxy Cookies Trash Help & support

Status Report Menjadi Claimed

The screenshot shows the REST Client interface with a sidebar on the left containing a list of APIs. The main panel displays a GET request to `http://localhost:3000/reports/4`. The response is a JSON object with details about the report, including `"id": 4`, `"item": "Rokok Surya"`, `"location": "Loken"`, `"description": "pas ditinggal sebentar sisa 1 batang"`, `"status": "Claimed"`, `"image": ""`, `"userId": 3`, `"createdAt": "2026-06-10T07:52:05.185Z"`, `"updatedAt": "2026-06-10T07:56:021Z"`, and a `"user"` object with `"fullName": "HANDOKO"`. The status bar indicates a successful response (200) and the request passed validation.

APIs: main

Default module

Endpoints

- PBS (9)
- LostFound (13)
- Login (1)
- POST Access Token
- Regs (2)
- POST Register User
- GET Tampil Data Reg...
- Reports (3)
- POST Simpan Data re...
- DEL Hapus data report
- GET Tampil Data Rep...
- Dashboard (1)
- Claim (2)
- GET Claim
- PAT Reject Claim

Need support? Contact us to get support.

Request: GET `http://localhost:3000/reports/4`

Params: Body 1 Headers Cookies Auth 1 Pre Processors Post Processors 1 Settings

none form-data x-www-form-urlencoded JSON XML Text Binary GraphQL msgpack application/json

Auto-generate Insert Dynamic Value Beautify

```
1 {
2   "status": "Claimed"
3 }
```

Body Cookies Headers 9 Console Actual F ... 200 83ms 570B Validation Success (200) Passed

Pretty Raw Preview Visualize JSON utf8

```
1 {
2   "id": 4,
3   "item": "Rokok Surya",
4   "location": "Loken",
5   "description": "pas ditinggal sebentar sisa 1 batang",
6   "status": "Claimed",
7   "image": "",
8   "userId": 3,
9   "createdAt": "2026-06-10T07:52:05.185Z",
10  "updatedAt": "2026-06-10T07:56:021Z",
11  "user": {
12    "fullName": "HANDOKO".
13  }
14 }
```

Design-first Mode Request-first Mode Online Request Proxy Cookies Trash Help & support

2. Membuat Dashboard Admin Menggunakan Data Asli

Sebelumnya dashboard admin masih menggunakan data dummy.

Yang saya kerjakan :

- Membuat endpoint dashboard summary
- Mengambil data langsung dari database menggunakan Prisma.
- Menampilkan jumlah report dan claim secara real-time.
- Menggunakan Prisma count untuk statistik dashboard.
- Melindungi endpoint dashboard agar hanya dapat diakses oleh admin.p

Endpoint:

GET /dashboard/summary

Data yang ditampilkan:

- Total Reports
- Total Claims
- Pending Claims
- Claimed Reports

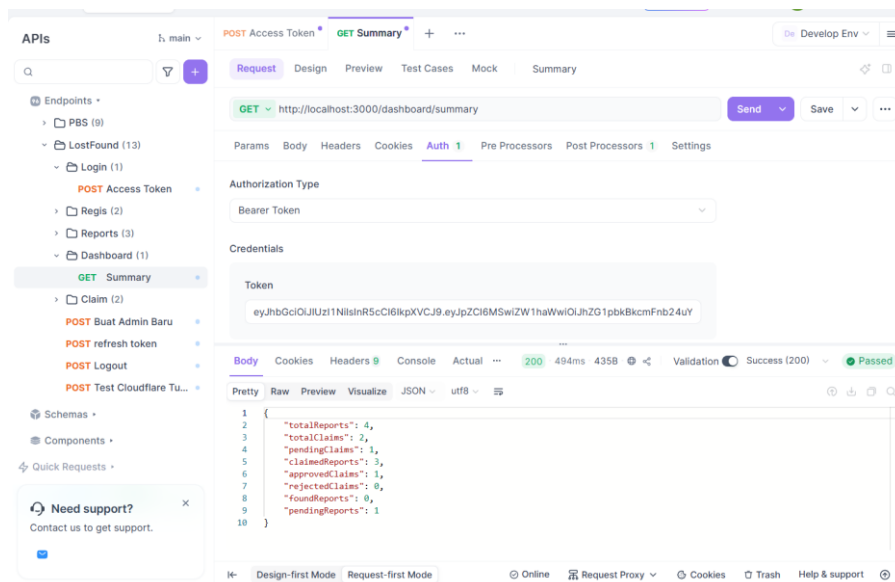
Keamanan:

- JWT Guard
- Roles Guard
- Admin Only

Hasil:

- Tanpa token -> 401 Unauthorized
- User biasa -> 403 Forbidden
- Admin -> 200 OK

Dashboard Summary API berhasil menampilkan data asli



The screenshot displays the Swagger UI interface for a REST API. The sidebar on the left lists various endpoints, including 'GET Summary' which is currently selected. The main area shows the configuration for this GET request, with the URL set to 'http://localhost:3000/dashboard/summary'. The 'Auth' tab is active, showing 'Bearer Token' as the authorization type. The bottom section displays the response details, including a 401 status code and the message 'Unauthorized'.

APIs

main

Endpoints

PBS (9)

LostFound (13)

Login (1)

Access Token

Regis (2)

Register User

Tampil Data Reg...

Reports (3)

Dashboard (1)

Summary

Claim (2)

Buat Admin Baru

refresh token

Logout

Test Cloudflare Tu...

Schemas

POST Access Token

GET Summary

GET Tampil Data Register

Develop Env

Request

Design

Preview

Test Cases

Mock

Summary

GET

http://localhost:3000/dashboard/summary

Send

Save

Params

Body

Headers

Cookies

Auth 1

Pre Processors

Post Processors 1

Settings

Authorization Type

Bearer Token

Credentials

Token

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6ImYyZWZ1aWwOIjoYmW5kb2t2QRy

Body

Cookies

Headers 9

Console

Act

403

15ms

400B

Pretty

Raw

Preview

Visualize

JSON

utf8

1

2

3

4

5

{

"message": "Akses ditolak! Fitur ini khusus untuk peran tertentu.",

"error": "Forbidden",

"statusCode": 403

}

Validation

Success

Copy for AI

Response Validation

HTTP Status Code should be 200

Need support?

Contact us to get support.

Design-first Mode

Request-first Mode

Online

Request Proxy

Cookies

Trash

Help & support

3. Final Security & Cleanup

Tujuan:

Meningkatkan keamanan backend sebelum demo.

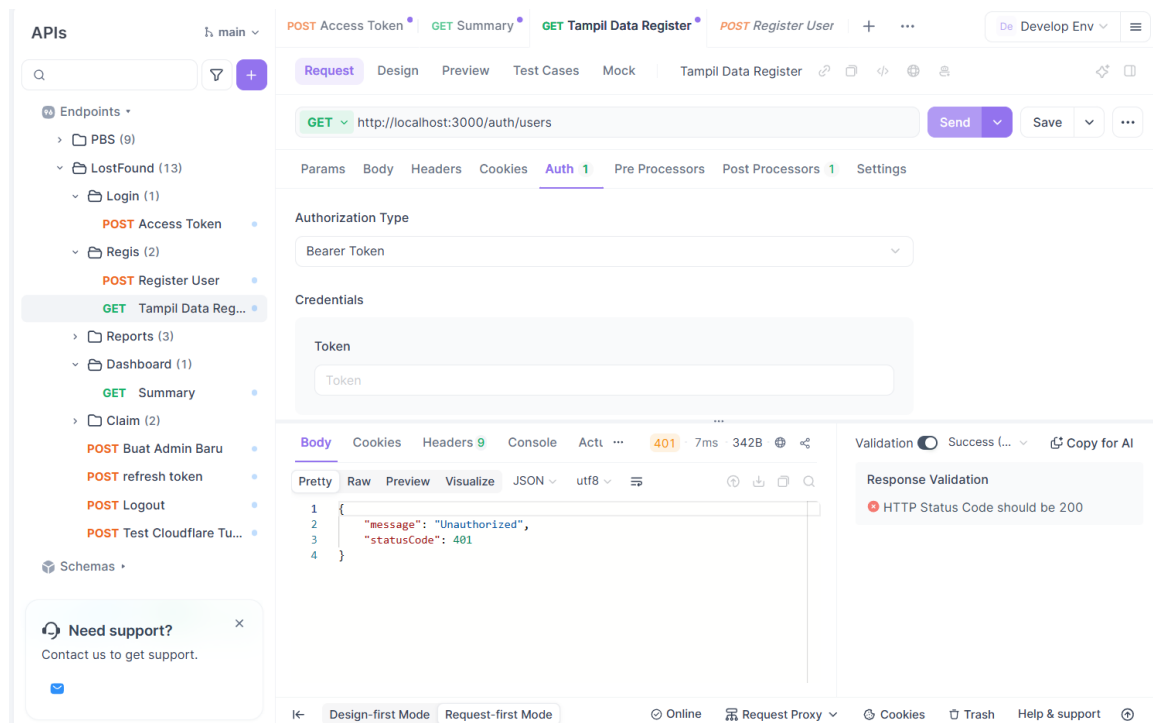
Pekerjaan:

- Mengamankan endpoint daftar user.
- Mengamankan endpoint create admin.
- Memperketat validasi upload gambar.
- Merapikan response login.
- Mencegah user enumeration.
- Menambahkan validasi email unik saat membuat admin.

Hasil:

- Endpoint admin lebih aman.
- Upload file lebih aman.
- Error response lebih konsisten.

Endpoint Admin Terlindungi : kondisi tanpa token 401 Unauthorized



Pakai token user 403 Forbidden (Ditolak)

The screenshot shows the Postman interface with a GET request to `http://localhost:3000/auth/users`. The request is configured with Bearer Token authorization. The response status is 403 Forbidden. The response body is a JSON object:

```
1 {
2   "message": "Akses ditolak! Fitur ini khusus untuk peran tertentu.",
3   "error": "Forbidden",
4   "statusCode": 403
5 }
```

The right sidebar shows the Response Validation section with a message: "HTTP Status Code should be 200".

Kondisi Pakai Token Admin (200) OK

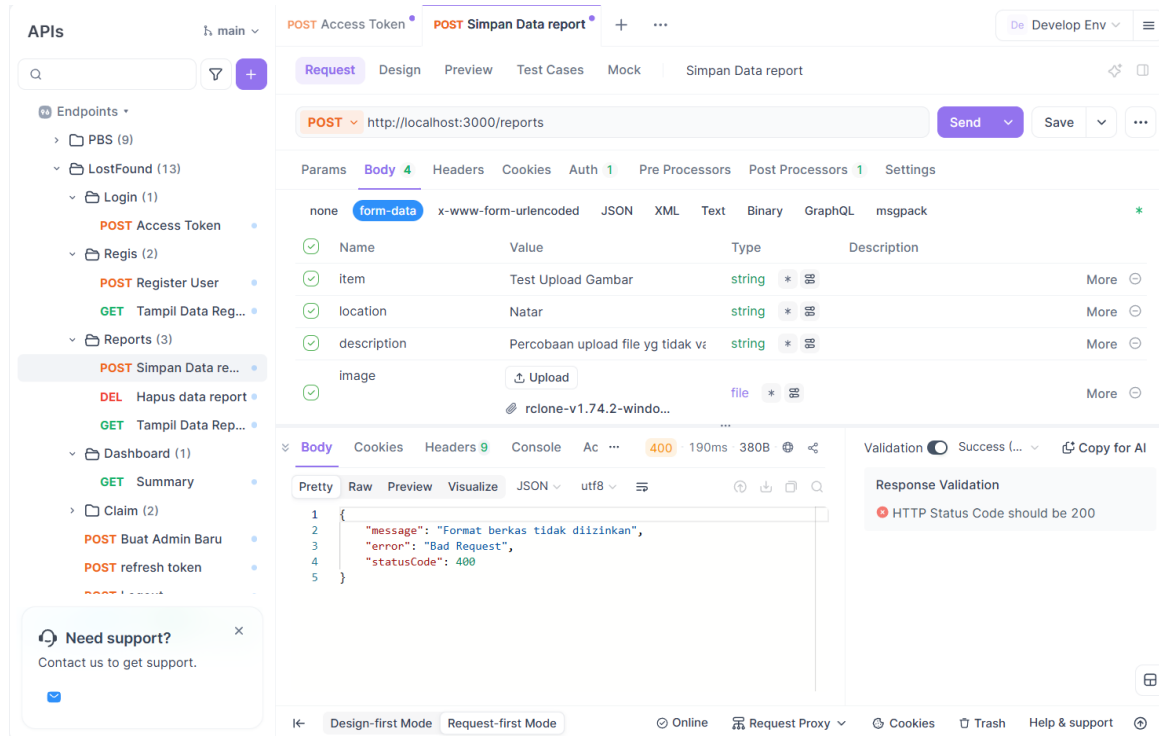
The screenshot shows the Postman interface with a GET request to `http://localhost:3000/auth/users`. The request is configured with Bearer Token authorization. The response status is 200 OK. The response body is a JSON array of two user objects:

```
1 [
2   {
3     "id": 3,
4     "fullName": "HANDOKO",
5     "email": "handoko@dragon.com",
6     "role": "user",
7     "createdAt": "2026-06-10T04:03:41.379Z"
8   },
9   {
10    "id": 1,
11    "fullName": "usergantenk",
12    "email": "admin@draeon.com"
13  }
14 ]
```

The right sidebar shows the Response Validation section with a message: "Response data differs from endpoint spec. 1. \$ should be object".

Validasi Upload Gambar

Backend menolak file dengan ekstensi tidak valid sehingga upload gambar lebih aman.



4. Pengujian

Pengujian dilakukan menggunakan Apidog.

Authentication:

- Login
- Refresh Token
- Logout

Authorization:

- 401 Unauthorized
- 403 Forbidden

Reports:

- Create Report
- Get Report
- Update Status
- Delete Report

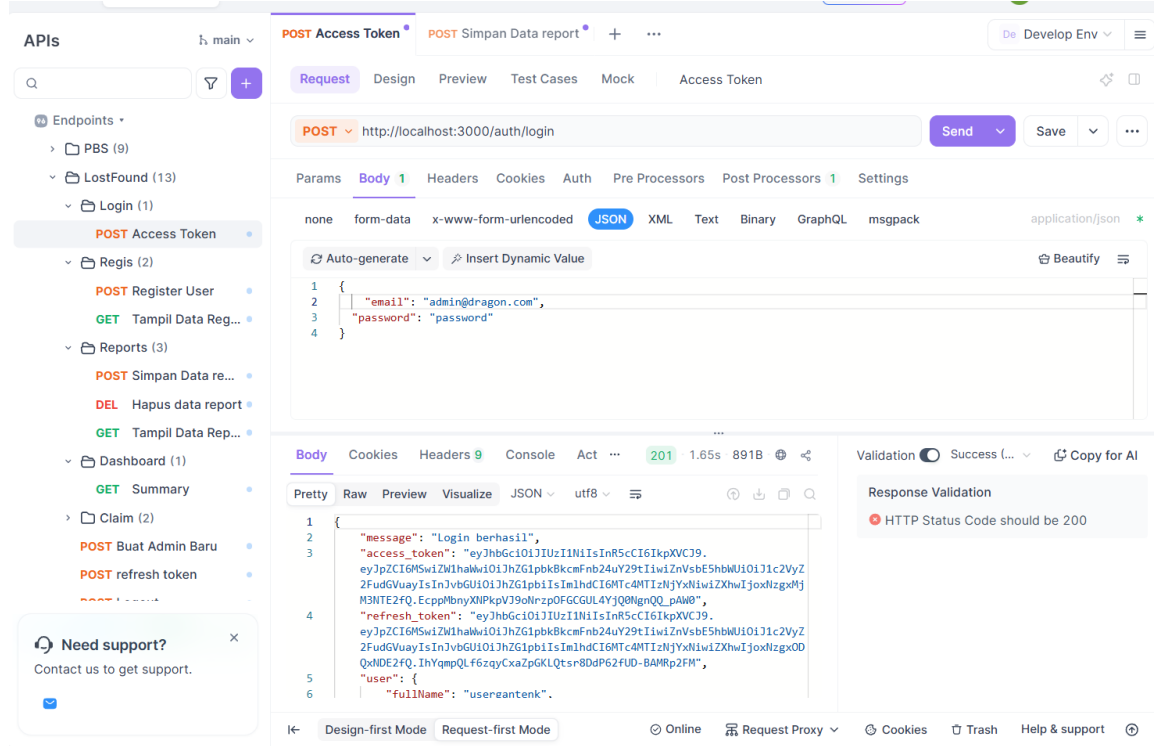
Claims:

- Create Claim
- Approve Claim
- Reject Claim

Dashboard:

- GET /dashboard/summary

Pengujian Login Berhasil dan Mendapatkan Token



5. Kendala dan Solusi

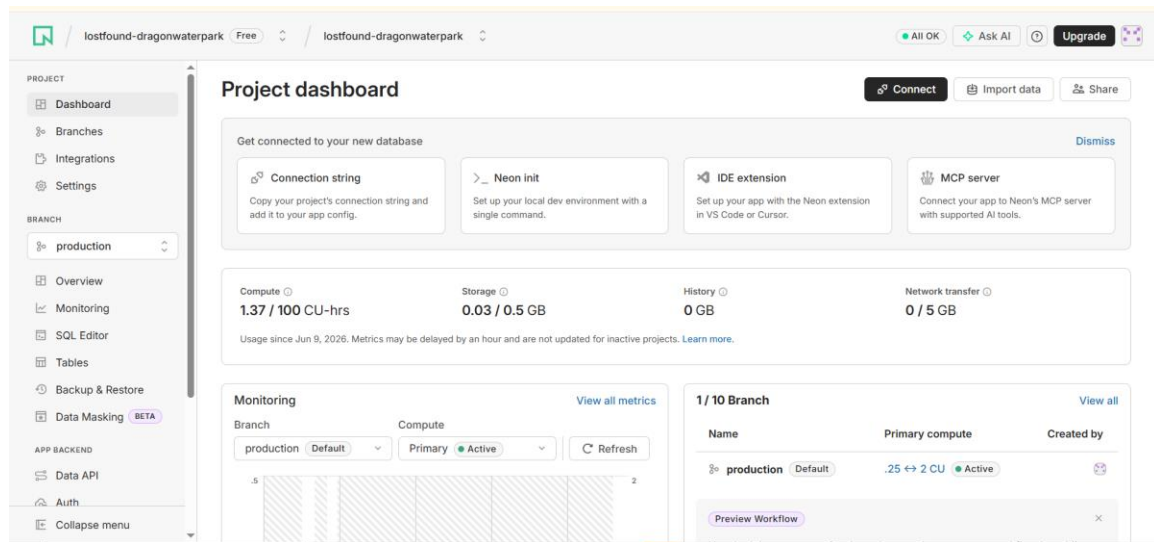
Kendala:

- Perbedaan database antara Prisma Studio dan backend.
- Integrasi frontend dan backend pada jaringan berbeda.
- Sinkronisasi data Neon PostgreSQL.

Solusi:

- Audit DATABASE_URL.
- Verifikasi Prisma Studio dan backend menggunakan database yang sama.
- Menggunakan Neon PostgreSQL.
- Menggunakan Cloudflare Tunnel untuk integrasi frontend dan backend.

Database Neon PostgreSQL



Cloudflare Tunnel untuk Integrasi Frontend dan Backend

```
PROBLEMS 9 OUTPUT DEBUG CONSOLE TERMINAL PORTS SPELL CHECKER 6
PS C:\cloudflared> cd C:\cloudflared
PS C:\cloudflared> .\cloudflared.exe tunnel --url http://localhost:3000
2026-06-12T03:10:57Z INF Thank you for trying Cloudflare Tunnel. Doing so, without a Cloudflare account, is a quick way to experiment and try it out. However, be aware that these account-less Tunnels have no uptime guarantee, are subject to the Cloudflare Online Services Terms of Use (https://www.cloudflare.com/website-terms/), and Cloudflare reserves the right to investigate your use of Tunnels for violations of such terms. If you intend to use Tunnels in production you should use a pre-created named tunnel by following: https://developers.cloudflare.com/cloudflare-one/connections/connect-apps
2026-06-12T03:10:57Z INF Requesting new quick Tunnel on trycloudflare.com...
2026-06-12T03:11:03Z INF +-----+
2026-06-12T03:11:03Z INF | Your quick Tunnel has been created! Visit it at (it may take some time to be reachable): |
2026-06-12T03:11:03Z INF | https://similarly-income-leaving-activation.trycloudflare.com |
2026-06-12T03:11:03Z INF +-----+
2026-06-12T03:11:03Z INF Cannot determine default configuration path. No file [config.yml config.yaml] in [~/cloudflared ~/.cloudflare-warp ~/cloudflare-warp]
```

7. Hasil Akhir Tahap 3

Pada akhir Tahap 3, bagian backend sudah berjalan dengan cukup stabil untuk kebutuhan demo. Beberapa fitur utama seperti alur klaim barang, dashboard summary, keamanan endpoint, validasi data, dan koneksi ke database Neon PostgreSQL sudah berhasil diselesaikan.

Backend juga sudah bisa diakses sementara menggunakan Cloudflare Tunnel agar lebih mudah diintegrasikan dengan frontend selama proses pengembangan.

Kesimpulan:

Pada Tahap 3 ini, saya fokus menyelesaikan bagian klaim barang, membuat data dashboard diambil langsung dari database, serta merapikan keamanan dan validasi backend. Dengan hasil tersebut, sistem sudah lebih siap untuk diuji dan didemonkan.